

PAIR & TUPLE

```
pair<int,int> p = {1, 2}; p.first; p.second;           // 1, 2
pair<int,int> p2 = make_pair(1,2);                   // alternative syntax
auto [a, b] = p;                                     // structured binding
// Pairs compare lexicographically by default
pair<int,int> x = {1,5}, y = {1,3};
x > y;                                                // true (1==1, compares 5 vs 3)
```

```
vector<pair<int,int>> vp;
vp.push_back({5, 0});                               // {value, index}
sort(vp.begin(), vp.end());                          // by .first, ties by .second
sort(vp.rbegin(), vp.rend());                       // descending
swap(p, p2);                                         // swap two pairs
```

```
tuple<int,int,int> t = {1, 2, 3};
get<0>(t);                                           // access by position
tie(a, b, c) = t;                                   // unpack tuple
```

//Index of the maximum element

```
int maxIdx = max_element(a.begin(), a.end()) - a.begin();
```

MAP / UNORDERED MAP

map = sorted by key, $O(\log n)$. unordered_map = no order, $O(1)$ average (faster).

```
map<int,int> mp;
mp[key]++;                                           // insert if new or increment
mp[key] = 5;                                         // directly set value
mp.at(key);                                          // no auto insert (throws if missing)
```

```
mp.count(key);                                       // 1 if exists, 0 if not
mp.find(key) != mp.end();                          // alternative existence check
mp.erase(key);                                       // remove by key
mp.erase(mp.find(key));                            // remove by iterator
```

```
for (auto &[k, v] : mp) { }                        // iterate (sorted key order)
for (auto it = mp.rbegin(); it != mp.rend(); it++) { } // reverse iterate
```

```
mp.size(); mp.empty(); mp.clear();
mp.lower_bound(key);                               // first key >= key
mp.upper_bound(key);                               // first key > key
mp.begin()->first;                                  // smallest key
mp.begin()->second;                                 // value of smallest key
mp.rbegin()->first;                                 // largest key
mp.rbegin()->second;                               // value of largest key
unordered_map<int,int> ump;                         // same syntax, unordered
map<string,int> wordCount;                          // string keys
map<pair<int,int>,int> grid;                        // 2D coordinate frequency
```

// Check if two arrays have the same frequency

```
map<int,int> cnt;
for (int x : a) cnt[x]++;
for (int x : b) cnt[x]--;
bool same = true;
for (auto &[k, v] : cnt)
    if (v != 0) same = false;
```